

HealthSync

Architecture, Concepts & Workflow Guide

A complete explanation of every concept used to build the project — the full-stack architecture, backend security and business logic, frontend state and routing, and the end-to-end workflows that tie them together.

Stack: React 19 · Vite · Tailwind CSS v4 · Express 5 · MongoDB · Mongoose 9 · JWT

Contents

1. Project Overview & Tech Stack
2. System Architecture & the Life of a Request
3. Backend — Layered Structure (Routes, Controllers, Services, Models)
4. Backend — The Express Middleware Pipeline
5. Backend — Error Handling Pattern (ApiError, ApiResponse, asyncHandler)
6. Backend — Data Modeling with Mongoose
7. Backend — Authentication: Registration, Email Verification & Login
8. Backend — Sessions: Access Tokens, Refresh Tokens & Rotation
9. Backend — Authorization, Validation & Security Hardening
10. Backend — Patient & Hospital Modules
11. Backend — Consent-Based Hospital-Patient Linking
12. Frontend — App Structure, Routing & Protected Routes
13. Frontend — Global Auth State (Context) & Session Restore
14. Frontend — The API Layer (fetch wrapper)
15. Frontend — Dashboard UI Patterns & React Hooks in Practice
16. Frontend — URL-Driven Tabs, Popovers & Recent Fixes
17. End-to-End Workflows (Step by Step)
18. API Endpoint Reference
19. Glossary of Key Concepts

1. Project Overview & Tech Stack

HealthSync is a full-stack healthcare platform with three kinds of accounts — **patients**, **hospitals**, and **admins** — each with its own dashboard. Patients manage their medical profile (personal details, address, emergency contact, profile photo); hospitals manage an institutional profile and build a patient list; and the two sides are connected through a consent-based linking system in which a hospital can only see a patient's full details after the patient explicitly approves the request from their own account.

The project is split into two independent applications that talk over HTTP:

Layer	Technology	Role
Frontend	React 19 + Vite + Tailwind CSS v4 + React Router 7 + lucide-react	Single-page application (SPA). Renders all UI in the browser, keeps auth state, and calls the API with fetch.
Backend	Node.js + Express 5, secured with helmet, express-rate-limit, express-validator, and more	Stateless REST API. Owns all business rules, validation, authentication and authorization.
Database	MongoDB via Mongoose 9	Document store. Collections: users, hospitals, admins, refreshtokens, hospitalpatients, auditlogs.

Why this separation matters: the frontend can never be trusted (anyone can open DevTools and send arbitrary requests), so every rule that matters — password checks, role checks, who may see which patient fields — is enforced on the server. The frontend's job is to make the legal operations pleasant to use; the backend's job is to make the illegal ones impossible.

2. System Architecture & the Life of a Request

Everything the app does follows the same round trip. Understanding this one path means you understand 90% of the project:

```

Browser (React SPA, localhost:3000)
  | fetch("http://localhost:5000/api/...", { Authorization: "Bearer <access token>",
  |                                           credentials: "include" -> refresh cookie })
  |
  v
Express app (app.js) -- middleware pipeline runs IN ORDER --
  requestId -> CORS -> helmet/sanitizers -> logging -> compression
  -> JSON body parser -> cookie parser -> static /uploads -> rate limiter
  v
Router (/api/auth | /api/patients | /api/hospitals)
  route-level middleware: rate limiter -> validators -> validate
                        -> authenticate (JWT) -> authorize (role)
  v
Controller (reads req, calls a service, shapes the HTTP response)
  v
Service      (ALL business logic lives here)
  v
Mongoose Model -> MongoDB
  v
ApiResponse { success, statusCode, message, data } ...or thrown ApiError
  v                                           v
JSON back to the browser                    Global errorHandler formats it

```

Two ideas are baked into this diagram. First, **statelessness**: the server keeps no session memory between requests; every request carries its own proof of identity (the JWT). That is what lets you restart the backend without logging everyone out. Second, **defense in depth**: a request must survive rate limiting, input validation, authentication, and role authorization before any business logic ever runs.

3. Backend — Layered Structure

The backend follows a strict layered (MVC-style) architecture. Each folder under Backend/src has exactly one responsibility:

Folder	Responsibility	Examples
routes/	Map URLs + HTTP verbs to middleware chains and controllers. No logic.	auth.routes.js, patient.routes.js, hospital.routes.js
controllers/	Translate HTTP to function calls: pull data off req, call a service, send an ApiResponse. Thin by design.	auth.controller.js, patient.controller.js
services/	The business layer. Every rule, DB query, token operation and side effect (emails, audit logs).	auth.service.js, patientLink.service.js, token.service.js
models/	Mongoose schemas: shape of the data, DB-level validation, virtuals, hooks, indexes.	User.js, Hospital.js, RefreshToken.js, HospitalPatient.js
middleware/	Reusable request interceptors.	authenticate.js, authorize.js, rateLimiter.js, errorHandler.js
validators/	express-validator rule sets for each endpoint's input.	auth.validator.js, patient.validator.js
utils/	Small shared helpers.	ApiError.js, ApiResponse.js, asyncHandler.js, logger.js
config/	Environment validation and tunable settings.	env.js, jwt.config.js, upload.config.js, db.js

Why layer it this way? Because each layer can change without breaking the others. You can swap how errors are formatted without touching business logic; you can unit-test a service without spinning up HTTP; and when you read a route file you see the entire security posture of an endpoint — limiter, validator, auth, role — declared on one line. A typical route reads like a sentence:

```
router.post('/login', authLimiter, loginValidator, validate, authController.login);  
router.use(authenticate, authorize('hospital')); // guards EVERY hospital route below it
```

4. Backend — The Express Middleware Pipeline

Middleware is the core Express concept: a function (`req`, `res`, `next`) that can inspect or modify a request, end it, or pass it on with `next()`. The app is literally a queue of these functions, and **order matters**. HealthSync registers them in this deliberate order in `app.js`:

#	Middleware	What it does and why it is at this position
0	<code>requestId</code>	Attaches a unique ID to every request first, so all later logs for one request can be correlated.
1	<code>cors</code>	Handles Cross-Origin Resource Sharing. Registered before everything else so that even error responses carry the <code>Access-Control-Allow-Origin</code> header — otherwise a crash deeper in the stack would surface in the browser as a misleading 'CORS error'. <code>credentials: true</code> allows the refresh cookie.
2	<code>helmet</code> + sanitizers	<code>helmet</code> sets defensive HTTP headers. A Mongo sanitizer strips keys starting with <code>\$</code> or containing dots (blocks NoSQL injection like <code>{"\$gt": ""}</code>). <code>hpp</code> collapses duplicate query params. <code>xss</code> cleans script payloads from input.
3	<code>morgan (dev)</code>	Logs each HTTP request during development.
4	<code>compression</code>	Gzips responses to cut payload size.
5	<code>express.json / urlencoded</code>	Parses JSON and form bodies into <code>req.body</code> (10 MB limit).
6	<code>cookieParser</code>	Parses the Cookie header into <code>req.cookies</code> — required to read the <code>HttpOnly</code> refresh token.
6. 5	<code>express.static('/uploads')</code>	Serves uploaded profile pictures as plain files.
7	<code>generalLimiter</code>	100 requests / 15 min / IP across the whole API.
9	<code>routes</code>	The actual /api endpoints.
10	<code>notFound</code>	Anything that matched no route becomes a clean 404.
11	<code>errorHandler</code>	Registered LAST: every thrown error from anywhere above lands here and leaves as consistent JSON.

5. Backend — Error Handling Pattern

HealthSync never writes error responses by hand inside route logic. Instead it uses three small utilities that together form one of the most important patterns in the codebase.

ApiError — throwable HTTP errors

A custom class extending `Error` that carries a status code and an optional list of field errors, with factory helpers so services read naturally: `throw ApiError.conflict('Email already registered')`, `ApiError.unauthorized(...)`, `ApiError.notFound(...)`. Services stay HTTP-aware but never touch `res` directly.

asyncHandler — no try/catch boilerplate

Express (before v5) did not forward errors thrown inside async functions. This 3-line wrapper fixes that once and for all — every controller is wrapped in it, so a rejected promise automatically flows into the global error handler:

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

Global errorHandler — one exit door

The final middleware translates every failure into the same JSON shape `{ success: false, message, errors }`. It also recognizes and rewrites third-party errors: a Mongoose **CastError** (malformed ObjectId) becomes 400; Mongo **duplicate key** (code 11000, e.g. registering an existing email) becomes 409 Conflict; Mongoose **ValidationError** becomes 400 with per-field messages; and JWT errors become 401. Stack traces are attached only in development.

ApiResponse — one success shape

The mirror image for success: every controller ends with `res.json(ApiResponse.ok(data, message))`, producing `{ success: true, statusCode, message, data }`. Because both success and failure have a fixed shape, the frontend needs exactly one response parser.

6. Backend — Data Modeling with Mongoose

Mongoose wraps raw MongoDB documents in **schemas** that add validation, computed fields, hooks and query helpers. The User model (which IS the patient — there is no separate Patient collection) demonstrates nearly every technique used in the project.

Schema-level validation

Every field declares its own rules: `required`, `minlength/maxlength`, `trim`, `lowercase`, `regex match` (Indian mobile numbers `/^[6-9]\d{9}$`, 6-digit pincodes), and `enum whitelists` for gender, blood group and link status. This is the last line of defense — even if a request validator were bypassed, bad data cannot be persisted.

`select: false` — secrets stay out of queries

Sensitive fields (`password`, `verification` and `reset tokens`) are marked `select: false`, so they are excluded from every query result unless explicitly requested with `.select('+password')` — which only the login code does. On top of that, a custom `toJSON()` deletes `password` and `lockout` fields before any document is serialized into a response.

Virtuals — computed, never stored

A **virtual** is a property computed on read. `user.age` is derived from `dob` at access time (so it can never go stale), and `user.patientId` formats the first 8 hex characters of the Mongo `_id` as the public ID patients share with hospitals:

```
userSchema.virtual('patientId').get(function () {
  return `HS-${this._id.toString().slice(0, 8).toUpperCase()}`;
});
```

Pre-save hook — passwords are hashed exactly once, in one place

A `pre('save')` hook `bcrypt`-hashes the password whenever it changes. No controller or service ever hashes manually, so it is impossible to accidentally store a plaintext password:

```
userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, bcryptSaltRounds);
});
```

Instance & static methods — behavior lives with the data

Instance methods: `comparePassword()` (`bcrypt compare`), `isLocked()`, `incrementLoginAttempts()` (locks the account after N failures using atomic `$inc/$set` updates), `resetLoginAttempts()`. Statics on `RefreshToken`: `saveToken()`, `findByRawToken()`, `revokeFamily()`, `deleteAllTokens()`.

Indexes

Three kinds appear in the project. **Unique indexes** on `email` and `mobile` prevent duplicate accounts at the database level. A **compound unique index** on `HospitalPatient { hospital: 1, patient: 1 }` guarantees a single link document per pair, so re-requests reuse the same record. A **TTL index** on `RefreshToken.expiresAt` makes MongoDB itself delete expired sessions — no cleanup job needed. `timestamps: true` auto-maintains `createdAt/updatedAt` everywhere.

7. Backend — Authentication: Registration, Email Verification & Login

7.1 Registration

Registration creates the account and nothing else — it deliberately does **not** log the user in. The service checks for an existing email and mobile (returning 409 Conflict), creates the user (the pre-save hook hashes the password), fires an audit-log entry, and — when SMTP is configured — sends a verification email. Patients and hospitals register through separate endpoints because they are different collections with different required fields (hospitals need a registration number, type, and address up front).

7.2 Email verification — hash what you store

The server generates a random 32-byte token, emails the **raw** token to the user as a link, but stores only its **SHA-256 hash** with a 24-hour expiry. When the link is clicked, the frontend posts the raw token, the server hashes it again and looks up the match. If the database ever leaked, the stored hashes could not be turned back into working verification links. The same hash-what-you-store rule is applied to password-reset tokens and refresh tokens.

A pragmatic dev-mode fallback: if no SMTP credentials are configured, accounts are auto-verified and the email is logged to the console instead — so local development never blocks on email delivery, while production (and the test suite) enforce the full flow.

7.3 Login — brute-force resistant by design

Login pulls the user **with** the hidden fields (`.select('+password +loginAttempts +lockUntil')`) and then walks a fixed checklist:

Step	Check	On failure
1	Account exists for that email	401 'Invalid credentials' — deliberately the same message as a wrong password, so attackers cannot probe which emails are registered.
2	Account is not locked (lockUntil in the future)	403 'Account locked' + audit event.
3	bcrypt compare of the password	Failed-attempt counter incremented atomically; after the configured max the account locks for a cooldown period. Audit event either way.
4	Email is verified (when enforcement is on)	403 'Email not verified'.
5	Success	Attempts reset; lastLogin, IP and device (parsed from the User-Agent with ua-parser-js) recorded; tokens issued.

On success the service returns three things: the sanitized user object, a short-lived JWT **access token** (returned in the JSON body), and a long-lived **refresh token** (set as an HttpOnly cookie). One `login(email, password, role)` dispatcher routes to the patient, hospital, or admin variant based on the role the user picked in the UI, so the same endpoint serves all three collections.

8. Backend — Sessions: Access Tokens, Refresh Tokens & Rotation

This is the most security-dense part of the project. The design splits identity into two tokens with opposite trade-offs:

	Access token	Refresh token
What it is	A signed JWT: { sub: userId, role, type } signed with a server secret (HMAC). Anyone can read it; nobody can forge or alter it without the secret.	40 random bytes (crypto.randomBytes) — meaningless by itself; a database key to a session document.
Lifetime	Minutes (short). Expiry is inside the signed payload.	Days. Enforced by the stored expiresAt + a Mongo TTL index.
Where it lives	Frontend memory / localStorage; sent as the Authorization: Bearer header.	An HttpOnly cookie — JavaScript can NEVER read it, which neutralizes token theft via XSS.
How verified	jwt.verify() — pure math, no DB hit. This keeps every authenticated request fast.	SHA-256 hash of the raw value looked up in the refreshtokens collection (O(1) via index).
Revocable?	No (that is why it must be short-lived).	Yes — delete or revoke the DB row and the session dies.

8.1 Token rotation and family-based reuse detection

Every call to POST /auth/refresh performs a **rotation**: the presented refresh token is marked isUsed and a brand-new one is issued in the same **family** (a UUID minted at login; one family = one device session). A refresh token is therefore valid for exactly one use.

The payoff is **replay-attack detection**: if a token that is already used or revoked is ever presented again, it means two parties hold the same token — the legitimate user plus, presumably, a thief. The server cannot tell which one is which, so it revokes the **entire family**, killing both copies of the session, logs a security warning, and forces a fresh login. Stolen refresh tokens thus have a one-use shelf life at most.

```
if (tokenDoc.isUsed || tokenDoc.isRevoked) {
  await tokenService.revokeFamilyTokens(tokenDoc.family); // kill the whole session line
  throw ApiError.unauthorized('Suspicious activity detected. Please log in again.');
```

8.2 Why SHA-256 for tokens but bcrypt for passwords?

bcrypt is intentionally slow to defeat brute-force guessing of **low-entropy** human passwords. Refresh tokens are 256 bits of pure randomness — unguessable by brute force — so slow hashing adds nothing, while fast SHA-256 enables indexed O(1) lookups. Right tool, right threat model.

8.3 Logout

POST /auth/logout deletes the presented refresh token's DB row and clears the cookie (this device only); /auth/logout/all deletes every token for the user (all devices). The access token simply ages out within minutes. Each RefreshToken row also stores the device, browser and IP it was created from — the groundwork for a 'manage my sessions' screen.

9. Backend — Authorization, Validation & Security Hardening

9.1 authenticate + authorize (RBAC)

`authenticate` reads the Bearer header, verifies the JWT signature and expiry, and attaches `req.user = { id, role }`. `authorize('hospital')` is a **higher-order middleware** — a function that returns a middleware closed over the allowed roles — and rejects with 403 if `req.user.role` is not in the list. Both are mounted router-wide (`router.use(authenticate, authorize('user'))`), so it is impossible to add a new patient route and forget to protect it. Note the distinction: 401 Unauthorized = "I don't know who you are"; 403 Forbidden = "I know who you are, and you may not do this".

9.2 Input validation (express-validator)

Each endpoint has a declarative rule set in `validators/` — email format, password policy, enum membership, ObjectId shape for URL params — followed by a shared `validate` middleware that collects violations and throws a 400 `ApiError` containing per-field messages **before the controller ever runs**. The frontend surfaces the first field message directly, which is why validation errors read nicely in the UI.

9.3 The hardening stack, threat by threat

Threat	Defense in HealthSync
Brute-force / credential stuffing	Tiered rate limits (5 login attempts / 15 min per IP; 3 registrations / hour) + per-account lockout after repeated failures.
NoSQL injection	Sanitizer strips \$-prefixed and dotted keys from body/query/params, so payloads like { email: { "\$gt": "" } } become inert. Schema type casting is a second wall.
XSS (stored/reflected)	<code>express-xss-sanitizer</code> cleans inputs; React escapes all output by default; the refresh token is unreachable from JS (<code>HttpOnly</code>).
Token theft / replay	Short-lived access JWTs; hashed, rotated, family-tracked refresh tokens (Section 8).
Password database leak	<code>bcrypt</code> hashes with per-password salts; verification/reset tokens stored only as SHA-256 hashes.
HTTP parameter pollution	<code>hpp</code> collapses duplicated query parameters.
Clickjacking / MIME sniffing etc.	<code>helmet</code> 's header set (<code>X-Frame-Options</code> , <code>nosniff</code> , and friends).
User enumeration	Identical 'Invalid credentials' message whether the email exists or the password is wrong.
Oversized payloads	10 MB body limit; <code>multer</code> restricts upload size and MIME types for profile pictures.

9.4 Auditing & logging

Security-relevant events (register, login, failures, lockouts, token refresh, link requests, approvals, discharges) are written to an **AuditLog** collection with user, action, IP, user-agent and metadata — an append-only trail of who did what, from where, when. Winston provides leveled application logs; morgan logs HTTP traffic in development; the `requestId` ties log lines to individual requests.

10. Backend — Patient & Hospital Modules

10.1 Allow-list profile updates

PATCH `/patients/profile` never passes `req.body` to the database directly. The service copies **only** fields on an explicit `ALLOWED_UPDATE_FIELDS` list (name, mobile, dob, gender, blood group, address.*, emergencyContact.*) into a Mongo \$set using dot-notation paths ('address.city' updates one nested field without replacing the whole address object). This blocks **mass assignment**: a malicious payload containing `isVerified: true` or `role: 'admin'` is silently ignored because those keys are not on the list. The hospital PATCH works the same way for its own field set, and immutable identity fields (email, registration number, hospital type) are simply not on any list.

10.2 File uploads (multer)

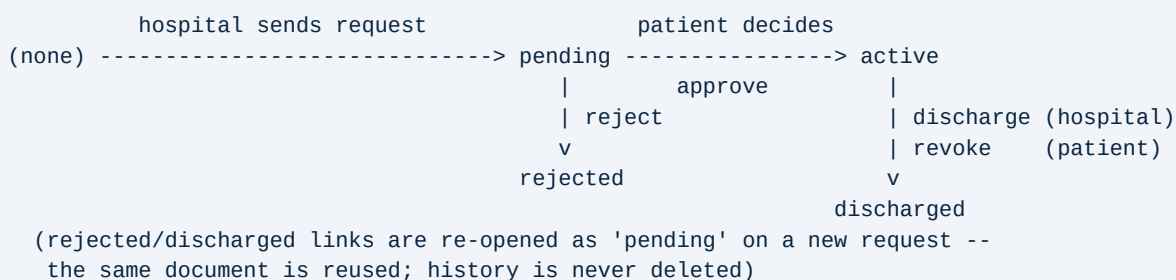
Profile pictures arrive as multipart/form-data handled by **multer**, configured with a disk destination, a size cap, and an image-only MIME filter. The database stores just the relative path; the file itself is served statically from `/uploads`. Replacing or deleting a photo also removes the old file from disk so orphans do not accumulate.

10.3 Dashboard summaries & profile completeness

GET `/patients/dashboard` and `/hospitals/dashboard` return the profile plus a summary object. **Profile completeness** is computed on the fly — the service counts how many of a fixed list of profile fields are filled and returns a rounded percentage — which the UI renders as progress bars and a nudge banner. Counts of linked patients and pending requests come from the linking module; appointment/record counts are placeholders for future modules.

11. Backend — Consent-Based Hospital-Patient Linking

The standout design decision in HealthSync: a hospital cannot simply 'add' a patient and read their data. Access is a **relationship with a lifecycle**, stored in its own `HospitalPatient` collection so that neither the User nor the Hospital document is modified, one patient can be linked to many hospitals, and every transition is timestamped and audited.



11.1 Privacy before consent — masked summaries

The service defines two projection functions. `preConsentSummary` (returned by lookup and while a request is pending) exposes only the public identity: patient ID, name, age/gender, verification badge, and a **masked** mobile number (•••••3210) so front-desk staff can visually confirm they found the right person.

`postConsentSummary` adds full mobile, email, blood group, DOB, address and emergency contact — and is only ever applied when `link.status === 'active'`. The filter lives in the serializer, so there is no code path that can leak full details for a non-active link.

11.2 Looking a patient up by public ID

Patients share their ID (HS-XXXXXXX) verbally — that is the first, informal consent signal. Because `patientId` is a virtual (not stored), the lookup matches it against the first 8 characters of `_id` with an aggregation-style `$expr` query; an email lookup is the fallback. Every lookup is audit-logged, and both sides' list endpoints compute per-status counts with a MongoDB **aggregation pipeline** (`$match -> $group -> $sum`) in parallel with the list query via `Promise.all`.

11.3 Ownership checks on every mutation

Every state change queries by **both** the link ID **and** the caller's own ID — `findOne({ _id: linkId, patient: userId })` — so a patient can only answer requests addressed to them and a hospital can only discharge its own links. This is authorization at the data layer, on top of the route-level role check, and it is what prevents IDOR (tampering with IDs in the URL to reach someone else's records).

12. Frontend — App Structure, Routing & Protected Routes

The frontend is a Vite-built React 19 SPA. `main.jsx` mounts `<BrowserRouter><AuthProvider><App/>`; `App.jsx` is a pure route table:

/	Landing	/dashboard/patient	(ProtectedRoute role="patient")
/login	Login (role toggle)	/dashboard/hospital	(ProtectedRoute role="hospital")
/signup	Signup	/dashboard/admin	(ProtectedRoute role="admin")
/verify-email	VerifyEmail	/admin/login	(separate, unadvertised)

Client-side routing means the browser never reloads between screens — React Router swaps components in place and manipulates the History API. Tailwind CSS v4 provides styling as utility classes directly in JSX (design tokens like the brand palette and fonts are declared in a CSS `@theme` block); lucide-react supplies the icons.

Route guarding

ProtectedRoute is a tiny wrapper component implementing three rules: while the session is still being restored it shows a loading state (preventing a redirect flash); if unauthenticated it redirects to the right login page, preserving the intended role as a query parameter; if authenticated with the **wrong** role (a patient opening /dashboard/hospital) it bounces to that user's own dashboard. All redirects use `replace` so dead ends never pollute Back-button history. This is UX-level protection only — the real enforcement is the backend's `authenticate/authorize` pair; a user who tampers with the client still cannot fetch any data.

13. Frontend — Global Auth State (Context) & Session Restore

Auth state (access token, user, role) is needed everywhere, so it lives in a **React Context** — AuthProvider owns the state, and any component reads it with the custom hook `useAuth()`. This avoids 'prop drilling' the token through every component layer.

Persistence and the two-token split, client side

The access token, user and role are mirrored to **localStorage** so a page refresh does not instantly log you out. The refresh token is deliberately **not** there — it exists only as the `HttpOnly` cookie the browser attaches automatically because every fetch uses `credentials: 'include'`. Worst case, an XSS attacker could read a 15-minute access token, but never the long-lived credential.

Silent session restore on page load

```

stored access token? --> GET /auth/me ----- valid --> session restored
      |                               |
      no                             expired/invalid
      v                               v
  POST /auth/refresh (HttpOnly cookie) --> new access token --> /auth/me --> restored
      |
      fails --> logged out, show public pages
  
```

This is why closing the tab and coming back keeps you signed in without ever storing a password. A loading flag guards the whole app while restore runs (ProtectedRoute waits on it), and a role-mapping layer translates the backend's 'user' role to the UI's friendlier 'patient' at this single boundary. Login stores the returned token in state; logout calls the API best-effort and always clears local state, so the UI can never get stuck logged in.

14. Frontend — The API Layer (fetch wrapper)

All HTTP goes through one function, `request(path, options)` in `lib/api.js`, with the endpoints grouped into `authApi`, `patientApi` and `hospitalApi`. Centralizing this buys five things:

Concern	How the wrapper handles it
Auth header	If a token is passed, it becomes <code>Authorization: Bearer</code> automatically.
Cookies	credentials: 'include' on every call, so the <code>HttpOnly</code> refresh cookie flows to <code>/auth/refresh</code> .
Content types	JSON bodies get <code>Content-Type: application/json</code> ; for <code>FormData</code> uploads the header is deliberately omitted so the browser can set the multipart boundary itself.
Error normalization	Network failures, non-JSON responses, HTTP error statuses and <code>{ success:false }</code> payloads are ALL converted into a thrown <code>Error</code> whose message is the backend's message (or first field-level validation error). UI code just <code>try/catches</code> and shows <code>err.message</code> .
Config	The base URL comes from the <code>VITE_API_URL</code> env var with a <code>localhost</code> fallback; <code>API_ORIGIN</code> (base minus <code>/api</code>) resolves statically served upload URLs.

Because the backend's `ApiResponse`/`ApiError` guarantee one response shape, this one parser covers every endpoint in the app — a direct payoff of the backend's consistency.

15. Frontend — Dashboard UI Patterns & React Hooks in Practice

Container / presentational split

Each dashboard page (`PatientDashboard`, `HospitalDashboard`) is a **container**: it owns state, data fetching and navigation, and passes plain props down to presentational pieces — `DashboardView`, `PatientsView`, `SettingsView`, `ProfilePanel`, `StatCard`, `InfoRow`, `EmptyState`, `Field`. The small shared pieces keep the two dashboards visually and behaviorally consistent without duplication.

The hooks the project actually uses, and why

Hook	Where and why
<code>useState</code>	Every piece of interactive state: active tab data, form fields, loading/error/success flags, modal and panel visibility, busy-row IDs during row-level API calls.
<code>useEffect</code>	Data fetching on mount and when dependencies change. Every fetch effect uses the cancelled-flag cleanup pattern: a variable flipped in the cleanup function prevents <code>setState</code> after unmount (stale responses from abandoned screens are ignored).
<code>useCallback</code>	Stabilizes fetch functions (refresh, load, loadNotifications) so they can safely appear in dependency arrays and be passed to children without retriggering effects every render.
<code>useMemo</code>	Memoizes the <code>AuthContext</code> value object so consumers only re-render when auth data actually changes.
<code>useContext</code>	<code>useAuth()</code> — reads the global session anywhere.
<code>useRef</code>	The hidden file input for profile photo upload: a button forwards clicks via <code>fileInputRef.current.click()</code> .
<code>useNavigate</code> / <code>useSearchParams</code>	Programmatic navigation after login/logout; URL-driven dashboard tabs (Section 16).

Controlled forms and minimal PATCH payloads

Every input is a **controlled component** — its value lives in React state and `onChange` writes back — so validation, prefilling and resets are trivial. On save, a `payloadFromForm()` helper trims each field and includes only non-empty values, building the smallest possible PATCH; the backend's allow-list is its server-side mirror. Edit forms are initialized from the current profile with the lazy form (`useState(() => formFromUser(profile))`) so the mapping runs once, not on every keystroke.

Honest UI states

Every data-driven view renders exactly one of: a loading state (spinner), an error state with a retry action, an **empty state** that explains what will appear and how to get there (no fake placeholder data anywhere), or the data itself. After mutations (approve link, save profile), success notices render dismissibly and the dashboard summary is quietly re-fetched (`refresh().catch(() => {})`) so counters stay accurate without blocking the interaction. Row-level busy tracking (`busyId`) disables just the button being acted on, and modals/slide-overs (Add Patient, Profile panel) close on backdrop click and reset their internal state each time they open.

16. Frontend — URL-Driven Tabs, Popovers & Recent Fixes

16.1 Why the Back button used to jump to the login page

Originally the active sidebar tab was plain component state (`useState( 'dashboard' )`). Switching tabs re-rendered the page but wrote nothing to the browser's **History API** — so from the browser's point of view you had never left `/dashboard/hospital`, and pressing Back popped the previous real entry: the login page.

The fix promotes the tab into the URL with React Router's `useSearchParams`. The active tab is now **derived** from `?tab=` (validated against the known tab list, defaulting to `dashboard`), and switching tabs calls `setSearchParams({ tab })`, which pushes a history entry. Back and Forward now walk through your actual tab history, tabs are bookmarkable and survive refresh — the URL has become the single source of truth for navigation state. Complementing this, login navigates with `replace: true`, removing the login page from history entirely once a session starts.

```
const [searchParams, setSearchParams] = useSearchParams();
const activeSidebarItem = TAB_KEYS.includes(searchParams.get("tab"))
  ? searchParams.get("tab") : "dashboard";
const goTo = (item) => { if (item !== activeSidebarItem) setSearchParams({ tab: item }); };
```

16.2 Notification & Help popovers

The topbar bell and help buttons are a shared `TopbarActions` component used by both dashboards. Notifications are **derived data** — no new backend was needed: the component maps existing link records into feed items (hospital side: pending / approved / declined patient requests; patient side: incoming hospital requests and new links), sorts them newest-first, and shows a real unread count instead of a hard-coded dot. The list is re-fetched each time the dropdown opens, and clicking an item deep-links to the relevant tab via the same `goTo` mechanism. The Help popover offers a support email and an enquiry box that opens the user's mail client via a `mailto:` URL — an honest stand-in until an enquiry endpoint exists.

16.3 Overflow-proof positioning

The panels are positioned `position: fixed` just below the header at the viewport's right edge — fixed elements are laid out relative to the viewport, so they can never widen the document or trigger stray scrollbars. Their size is clamped (`max-h-[calc(100vh-5.5rem)]`, `max-w-[calc(100vw-2rem)]`) with internal scrolling for long lists, an invisible full-screen backdrop button closes them on outside click, and the dashboard root uses `h-screen overflow-hidden` so only the content column scrolls — never the window.

17. End-to-End Workflows (Step by Step)

17.1 Sign-up to verification to first login

1. User fills Signup (role toggle: Patient | Hospital) -> POST /auth/register[/hospital]
2. Backend: validate -> uniqueness checks -> create account (password bcrypt-hashed by the pre-save hook) -> store SHA-256 hash of a verification token -> email raw link
3. Frontend redirects to /login (replace:true) with a "check your email" notice
4. User clicks the email link -> /verify-email?token=... -> POST /auth/verify-email
-> backend hashes the token, matches, checks the 24h expiry, sets isVerified
5. Login: POST /auth/login { email, password, role }
-> lockout check -> bcrypt compare -> verification check
-> JWT access token (JSON body) + refresh token (HttpOnly cookie)
6. AuthContext stores { accessToken, user, role } -> navigate(dashboard, replace:true)

17.2 An authenticated request, and what happens when the token expires

```
patientApi.getDashboard(accessToken)
  -> GET /api/patients/dashboard (Bearer header + cookies)
  -> pipeline -> authenticate (jwt.verify) -> authorize('user')
  -> controller -> service (profile + completeness + counts) -> ApiResponse

...15 minutes later the access token expires -> a request returns 401
  -> on next app load, AuthContext's silent restore calls POST /auth/refresh
  -> cookie token found by SHA-256 hash -> not used/revoked/expired?
  -> ROTATE: old marked used, new cookie issued (same family) -> new access JWT
  -> reuse of the old token later would revoke the entire family (Section 8)
```

17.3 Hospital adds a patient (the consent loop)

```
HOSPITAL  Patients tab -> Add Patient -> enters HS-XXXXXXX or email
          POST /hospitals/patients/lookup -> masked preview (.....3210)
          POST /hospitals/patients -> link { status: 'pending' }
PATIENT   Bell shows "wants to add you" -> My Hospitals card -> Approve / Decline
          PATCH /patients/hospitals/:linkId/respond { action }
          (ownership check: findOne({ _id: linkId, patient: ME }))
HOSPITAL  list now serializes the link with postConsentSummary -> full contact,
          blood group, DOB, address, emergency contact
LATER     hospital: PATCH ../discharge or patient: PATCH ../revoke
          -> status 'discharged', endedBy recorded, document kept forever
```

17.4 Profile edit

```
Edit Profile -> form prefilled from current profile (controlled inputs)
Save -> payloadFromForm() strips empty fields -> PATCH /patients/profile
Backend: validators -> allow-list -> $set dot-notation update -> sanitized JSON back
UI: success banner -> silent refresh() re-fetches completeness % (progress bar moves)
```

18. API Endpoint Reference

Auth — /api/auth

Endpoint	Guard	Purpose
POST /register	3/hour limit	Create patient account, send verification email
POST /register/hospital	3/hour limit	Create hospital account
POST /login	5/15min limit	Issue access JWT + refresh cookie (role-dispatched)
POST /refresh	cookie	Rotate refresh token, new access JWT, reuse detection
POST /verify-email	5/hour limit	Confirm email via hashed token
POST /resend-verification	5/hour limit	Re-issue verification email
POST /forgot-password	3/hour limit	Email a hashed, expiring reset token
POST /reset-password	3/hour limit	Set new password via reset token
POST /logout	JWT	Delete this device's refresh token
POST /logout/all	JWT	Delete all refresh tokens (every device)
POST /change-password	JWT + limit	Change password while logged in
GET /me	JWT	Current account profile (session restore)

Patients — /api/patients (JWT + role 'user')

Endpoint	Purpose
GET/PATCH /profile	Read / allow-list update of the patient profile
POST/DELETE /profile/picture	Upload (multer, image-only) / remove profile photo
GET /dashboard	Profile + completeness % + counts
GET /hospitals	My links (pending = incoming requests, active = my hospitals)
PATCH /hospitals/:linkId/respond	Approve or reject a pending hospital request (consent step)
PATCH /hospitals/:linkId/revoke	Revoke an active hospital's access

Hospitals — /api/hospitals (JWT + role 'hospital')

Endpoint	Purpose
GET/PATCH /profile	Read / allow-list update of the hospital profile
GET /dashboard	Profile + completeness % + patient/request counts
POST /patients/lookup	Find a patient by HS-ID or email — masked preview only
POST /patients	Send (or re-open) a link request -> status 'pending'
GET /patients	All links with per-status counts (aggregation pipeline)
PATCH /patients/:linkId/discharge	Discharge an active patient / withdraw a pending request

19. Glossary of Key Concepts

Term	Meaning in this project
SPA	Single-Page Application — one HTML page; React re-renders views client-side; navigation never reloads the browser.
REST API	Stateless HTTP endpoints organized around resources (profiles, links, sessions) using GET/POST/PATCH/DELETE semantics.
Middleware	An Express function (req, res, next) in an ordered chain; each can inspect, modify, reject or forward the request.
JWT	JSON Web Token — a signed (not encrypted) payload the server can verify without a database hit; used for 15-minute access tokens.
HttpOnly cookie	A cookie JavaScript cannot read; the browser attaches it automatically. Home of the refresh token; immune to XSS theft.
Token rotation / family	Every refresh burns the old token and issues a new one in the same family; a replayed old token kills the whole family.
bcrypt vs SHA-256	Slow, salted bcrypt for guessable passwords; fast SHA-256 for unguessable high-entropy tokens needing indexed lookup.
RBAC	Role-Based Access Control — <code>authorize('user' 'hospital' 'admin')</code> gates whole route groups after authentication.
401 vs 403	401 = not authenticated (unknown identity); 403 = authenticated but not permitted (wrong role, locked, unverified).
Mass assignment	Attack where extra JSON keys (role, isVerified) ride along an update; blocked by allow-list field copying.
IDOR	Insecure Direct Object Reference — guessing other people's IDs in URLs; blocked by owner-scoped queries (<code>findOne</code> with caller's ID).
NoSQL injection	Sending Mongo operators (<code>{'\$gt:'}</code>) as input; blocked by key sanitization + schema casting.
Rate limiting	Capping requests per IP per window (100/15min general; stricter on auth) to blunt brute force and abuse.
Account lockout	Per-account failed-attempt counter that temporarily locks logins — complements per-IP rate limits.
Virtual (Mongoose)	A computed property (age, patientId) evaluated on read, never stored, so it cannot go stale.
Pre-save hook	Model-level code that runs before persistence — the single place passwords are hashed.
TTL index	MongoDB index that auto-deletes documents at <code>expiresAt</code> — expired sessions clean themselves up.
Aggregation pipeline	Mongo's staged query language (<code>\$match/\$group/\$expr</code>) used for status counts and the patientId prefix lookup.
Audit log	Append-only record of security events (who, what, when, from where) in its own collection.
React Context	Built-in dependency injection for global state (the auth session) without prop drilling.
Controlled component	A form input whose value is React state — the basis of all forms in the app.
Derived state	Values computed from existing data at render time (active tab from the URL; notifications from link records) instead of stored copies that could drift.
Optimistic/quiet refresh	After a mutation, summary data is re-fetched in the background without blocking the UI.

Term	Meaning in this project
Empty state	A deliberate 'nothing here yet' design explaining what will appear — honesty instead of placeholder data.
History API / replace	pushState adds a Back-button entry (tab changes); replace overwrites one (login -> dashboard) so dead ends vanish.
CORS	Browser policy controlling cross-origin calls; the API allows exactly the frontend origin, with credentials, headers and methods enumerated.

Generated July 22, 2026 — reflects the codebase as of this date, including the URL-driven tab navigation and topbar popover fixes.